



Học ở trường

Nguyên tắc chung

1. Đảm bảo tính bí mật (Confidentiality): đảm bảo tài sản không thể bị truy nhập trái phép bởi những người không có thẩm quyền.
2. Đảm bảo tính nguyên vẹn (Integrity): đảm bảo tài sản không thể bị sửa đổi, bị làm giả bởi những người không có thẩm quyền.
3. Tính khả dụng, hay sẵn dùng (Availability): đảm bảo tài sản là sẵn sàng để đáp ứng sử dụng cho người có thẩm quyền

Các thuật ngữ

▼ Thế nào là Crypto?

- Là môn xây dựng và phân tích các giao thức mật mã để đạt được các mục tiêu về an toàn thông tin.
- Các nhà mật mã học phân tích các giao thức dưới các mô hình tấn công

- Cryptography: là môn học về các kỹ thuật toán học nhằm cung cấp các dịch vụ an toàn thông tin.
- Một giao thức (hoặc cơ chế) (protocol): là một bộ các thủ tục cho biết các bên tham gia cần phải làm những gì
- Cryptanalysis: là môn học về các kỹ thuật toán học nhằm phá huỷ các dịch vụ an toàn thông tin.
- Cryptology: là môn học bao gồm cryptography và cryptanalysis.
- Plaintexts: bản rõ
- Ciphertexts: bản mã
- Bản dịch : các kí tự đã được dịch đi 1 số đơn vị trong bảng chữ cái
- Keys: khoá
- Encryption: mã hoá
- Decryption: giải mã

Hệ thống mật mã khóa bí mật (Secret-key cryptography)

- Còn có tên là : mật mã khóa đối xứng - **symmetric key cryptosystem**
- Quá trình mã hóa và giải mã dùng cùng một khóa ($Z = Z'$), $\rightarrow$ mật mã khóa đối xứng
- đều là khóa cần phải giữ bí mật $\rightarrow$ mật mã khóa bí mật
- Vấn đề phân phối khóa: làm sao để chia sẻ khóa giữa A và B ?

Mật mã khóa công khai: Public-key cryptography(Còn có tên là mật mã khóa phi đối xứng)

- Khóa dùng để mã hóa và giải mã là khác nhau $Z \neq Z'$
- Tốn chi phí hơn khóa đối xứng

Phá mã: có nhiều mức

- Có rất nhiều kiểu tấn công, phụ thuộc vào:
 - Kiểu thông tin mà kẻ tấn công có thể có

- Tương tác với máy mã hóa
- Năng lực tính toán của kẻ tấn công
- **Tấn công chỉ dựa vào bản mã (Ciphertext-only attack):**
 - Kẻ tấn công **chỉ biết bản mã**
 - Mục tiêu: tìm được bản rõ và khóa
 - Chú ý: Một hệ thống bị tấn công bởi kiểu tấn công này thì hoàn toàn không an toàn
- **Tấn công dựa vào bản rõ (Known-plaintext attack):**
 - Kẻ tấn công biết một vài bản mã và các bản rõ tương ứng
 - Mục tiêu: Tìm ra khóa đã được dùng để mã hóa. Hoặc tìm ra cách giải mã các gói tin cùng khóa với các gói đã bắt được
- **Tấn công bản rõ có chọn lựa (Chosen-plaintext attack)**
 - Kẻ tấn công có thể chọn một số các bản rõ và nhận được các bản mã tương ứng.
 - Mục tiêu: Suy đoán khóa
- **Tấn công bản mã có lựa chọn (Chosen-ciphertext attack)**
 - Tương tự như trên nhưng kẻ tấn công có thể chọn một vài bản mã và nhận được các bản rõ tương ứng
 - Sự lựa chọn của bản mã có thể thay đổi tùy vào bản rõ nhận được trước đó

Một số mật mã cổ điển

▼ Mã dịch Shift cipher (mã cộng additive cipher)

- Dùng khóa K cho trước
- Với mỗi ký tự của bản rõ P được mã hóa thành ký tự thứ K sau nó (dịch đi K bước về phía phải):
 - $Y = (X \oplus K) \bmod 26 \Rightarrow$ Mã cộng
- **Giải mã:** Dịch trái K bước về phía trái

- Vd: I love you → L oryh brx
- **Cách tấn công : Thử dịch trái ⇒ thử 26 lần là ra**

▼ Mật mã 1 bảng thế (Monoalphabetic cipher)

Mỗi kí tự trong bảng chữ cái của bản rõ sẽ được hoán vị bởi một kí tự khác trong bản mã → khóa trong một cipher loại này là một bảng hoán vị

a	b	c	d	e	...	x	y	z
F	G	N	T	A	...	K	P	L

- Để phá loại mã này, chúng ta cần thu được nhiều bản tin với số lượng lớn chữ, kết hợp với kĩ thuật thống kê (ví dụ trong tiếng anh chữ cái e xuất hiện nhiều nhất) để truy ra được các quy luật thay thế (ví dụ nếu a xuất hiện nhiều nhất thì tức là a được hoán vị với e).

▼ Mật mã đa bảng thế (polyalphabetic cipher)

- Khóa sẽ quyết định thứ tự hòa trộn của các bảng như thế nào
- Sử dụng nhiều bảng thế, có một loại mã phổ biến : **mã Vigenre**
- Giả sử m là một số nguyên dương, $P=C=(X_{26})^m$, và $K = (k_1, k_2, \dots, k_m)$ là khóa, thế thì:
- Nếu K có ít chữ cái hơn bản mã, thì nhân K lên cho đến khi bằng :v
- **Thuật toán mã hóa:**
 - $e_k(p_1, p_2, \dots, p_m) = (p_1 + k_1, p_2 + k_2, \dots, p_m + k_m) \pmod{26}$
- **Thuật toán giải mã:**
 - $d_k(c_1, c_2, \dots, c_m) = (c_1 - k_1, c_2 - k_2, \dots, c_m - k_m) \pmod{26}$
- **Phá mã:**
 - Mấu chốt: làm sao tìm ra độ dài p của khóa
 - ⇒ Sử dụng **chỉ số trùng nhau** (index of coincidence : IC)
 - Xác suất để 2 thành phần ngẫu nhiên của một chuỗi có độ dài n là trùng nhau

- Công thức tính IC:
- $$IC(x) = \frac{\sum_{i=0}^{25} f_i(f_i - 1)}{n(n-1)}$$
- Trong đó f_i là tần số xuất hiện của kí tự alphabet thứ i ở trong x .
- **Tần số xuất hiện của các kí tự càng bằng nhau thì chỉ số IC càng nhỏ, ngược lại khi các kí tự có tần số lệch nhau**
- **Chỉ số IC của ngôn ngữ tự nhiên : 0.065**
- $\Rightarrow$ Tìm p sao cho chỉ số IC của các nhóm là lớn nhất (xấp xỉ 0.065)
- **Phá mã theo kiểu Kasiski:**
 - Tìm một cụm từ (cụm từ này không cần có nghĩa vì đã được mã hóa) mà tần suất xuất hiện nhiều trong bản mã, sau đó đánh dấu các vị trí xuất hiện $\Rightarrow$ sau đó lấy ước số chung lớn nhất của các vị trí xuất hiện của cụm từ này $\rightarrow$ đặt là $P \Rightarrow$ suy ra **độ dài m của khóa**
 - Tìm độ dài khóa bằng cách thử như phép IC nhưng chỉ thử các độ dài nhỏ hơn hoặc bằng P

Mật mã khối

- Mật mã khối là mật mã mà trong đó bản tin (plain text) được chia thành các khối có độ dài bằng nhau. Các khối này được mã hóa riêng biệt thành các khối mã có độ dài bằng khối đầu vào

key	000	001	010	011	100	101	110	111
0	001	111	110	000	100	010	101	011
1	001	110	111	100	011	010	000	101
2	001	000	100	101	110	111	010	011
3	100	101	110	111	000	001	010	011
4	101	110	100	010	011	001	011	111

Theo bảng này, dữ liệu plaintext 010100110111 sẽ được mã hóa thành:

010 100 110 111 $\rightarrow$ 111 011 000 101 theo key=1

010 100 110 111 $\rightarrow$ 100 011 011 111 theo key=4

▼ Điều kiện an toàn:

- Kích thước khối phải đủ lớn
 - để chống lại tấn công theo kiểu thống kê
 - $\Rightarrow$ gây ra độ trễ lớn
- Không gian khóa phải đủ lớn
 - để chống lại tìm kiếm vét cạn
 - Tuy nhiên lại gây ra khó khăn trong lưu trữ, phân phối

Nguyên tắc thiết kế

▼ Khuếch tán (diffusion)

- Khuếch tán đặc tính thống kê của bản tin vào bản mã
- Mỗi bit của bản tin và khóa phải ảnh hưởng lên nhiều bit của bản mã
- $\Rightarrow$ Mỗi bit của bản mã bị ảnh hưởng bởi nhiều bit của bản tin
- $\Rightarrow$ gây khó khăn trong việc phá mã dựa trên đặc tính thống kê của bản tin

– Tin: $T = t_1 t_2 \dots$

– Mã: $M_n = \left(\sum_{i=1}^k t_{n+i} \right) \bmod 26$

$\Rightarrow$ tần suất xuất hiện của các ký tự trong bản mã “gần giống nhau” hơn trong bản tin.

$\Rightarrow$ đặc tính về thống kê của các ký tự trong bản tin đã bị làm khuếch tán đi trong bản mã.

Phương pháp thực hiện : thực hiện nhiều lần thao tác: hoán vị + tác động thuật toán $\Rightarrow$ các bit tại các vị trí khác nhau của bản tin cùng tác động lên một bit của bản mã

▼ Hỗn loạn (Confusion)

- Sự phụ thuộc của bản mã đối với bản rõ phải thực phức tạp để gây rắc rối, cảm giác hỗn loạn đối với kẻ thù có ý định phân tích tìm qui luật để phá mã.
- Quan hệ hàm số của mã-tin tốt nhất là phi tuyến (non-linear)
- **Phương pháp thực hiện:** Thực hiện bằng các thuật toán thay thế phức tạp

▼ Mạng SPN

Mạng SPN

- Substitution and permutation network
 - Thuật toán sinh khoá (key schedule algorithm)
 - Input: khoá K
 - Output: N_r khoá con: $\{K^1, K^2, \dots, K^{N_r}\}$
 - Thuật toán mã hoá trong mỗi vòng lặp (round function)

- $g(w^{r-1}, K^r)$

$$\begin{aligned}w^0 &\leftarrow x \\w^1 &\leftarrow g(w^0, K^1) \\w^2 &\leftarrow g(w^1, K^2) \\&\vdots\end{aligned}$$

$$\begin{aligned}w^{N_r-1} &\leftarrow g(w^{N_r-2}, K^{N_r-1}) \\w^{N_r} &\leftarrow g(w^{N_r-1}, K^{N_r}) \\v &\leftarrow w^{N_r}\end{aligned}$$

Mạng SPN

- Độ dài bản tin: $l \times m$
 - l : độ dài của 1 khối
 - m : số khối
- Round function
 - Thay thế (Substitution): π_S
 - S-box: thay thế các khối l bit bằng l khác
 - Hoán vị (Permutation): π_P
 - Thay đổi vị trí của $l \times m$ bit
- Thuật toán mã hoá trong 1 vòng

$$\begin{aligned} u^r &\leftarrow w^{r-1} \oplus K^r \\ v^r &\leftarrow \pi_S(u^r) \\ w^r &\leftarrow \pi_P(v^r) \end{aligned}$$

Mạng SPN

Algorithm 3.1: SPN($x, \pi_S, \pi_P, (K^1, \dots, K^{N_r+1})$)

Số khoá con: $N_r + 1$

```

 $w^0 \leftarrow x$ 
for  $r \leftarrow 1$  to  $N_r - 1$ 
   $\left\{ \begin{array}{l} u^r \leftarrow w^{r-1} \oplus K^r \\ \textbf{for } i \leftarrow 1 \textbf{ to } m \\ \textbf{do } v_{\langle i \rangle}^r \leftarrow \pi_S(u_{\langle i \rangle}^r) \\ w^r \leftarrow (v_{\pi_P(1)}^r, \dots, v_{\pi_P(\ell m)}^r) \end{array} \right.$ 
 $u^{N_r} \leftarrow w^{N_r-1} \oplus K^{N_r}$ 
  for  $i \leftarrow 1$  to  $m$ 
     $v_{\langle i \rangle}^{N_r} \leftarrow \pi_S(u_{\langle i \rangle}^{N_r})$ 
 $y \leftarrow v^{N_r} \oplus K^{N_r+1}$ 
output ( $y$ )
  
```


- Ví dụ: $l = m = N^r = 4$

z	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$\pi_S(z)$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

z	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$\pi_P(z)$	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16

$K = 0011\ 1010\ 1001\ 0100\ 1101\ 0110\ 0011\ 1111$

$K^1 = 0011\ 1010\ 1001\ 0100$

$K^2 = 1010\ 1001\ 0100\ 1101$

$K^3 = 1001\ 0100\ 1101\ 0110$

$K^4 = 0100\ 1101\ 0110\ 0011$

$K^5 = 1101\ 0110\ 0011\ 1111$

Plain text $x = 0010\ 0110\ 1011\ 0111$

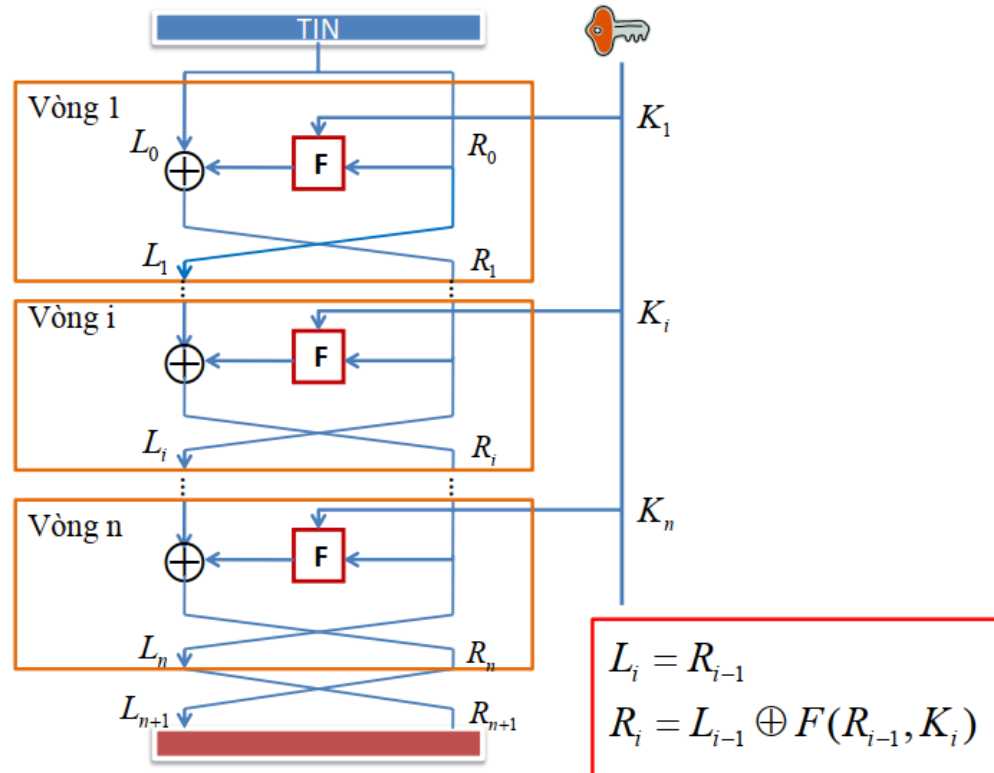
$w^0 = 0010\ 0110\ 1011\ 0111$
 $K^1 = 0011\ 1010\ 1001\ 0100$
 $u^1 = 0001\ 1100\ 0010\ 0011$
 $v^1 = 0100\ 0101\ 1101\ 0001$
 $w^1 = 0010\ 1110\ 0000\ 0111$
 $K^2 = 1010\ 1001\ 0100\ 1101$
 $u^2 = 1000\ 0111\ 0100\ 1010$
 $v^2 = 0011\ 1000\ 0010\ 0110$
 $w^2 = 0100\ 0001\ 1011\ 1000$

$K^3 = 1001\ 0100\ 1101\ 0110$
 $u^3 = 1101\ 0101\ 0110\ 1110$
 $v^3 = 1001\ 1111\ 1011\ 0000$
 $w^3 = 1110\ 0100\ 0110\ 1110$
 $K^4 = 0100\ 1101\ 0110\ 0011$
 $u^4 = 1010\ 1001\ 0000\ 1101$
 $v^4 = 0110\ 1010\ 1110\ 1001$
 $K^5 = 1101\ 0110\ 0011\ 1111,$
 $y = 1011\ 1100\ 1101\ 0110$

Cấu trúc Festel

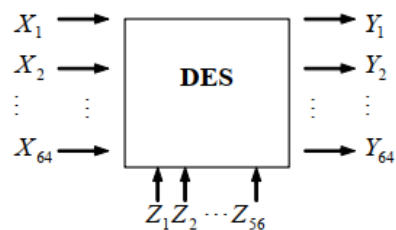
- Được sử dụng bởi phần lớn các mật mã khối hiện nay
- Bao gồm nhiều vòng lặp

- Tại mỗi vòng lặp sẽ thực hiện các thao tác hoán vị và thay thế
- Đầu vào của mỗi vòng lặp là đầu ra của vòng lặp trước đó và một khóa con được sinh ra từ khóa đầy đủ với thuật toán sinh khóa
- Giải mã là một quá trình ngược với các khóa con cho mỗi vòng sẽ được phát sinh theo thứ ngược



▼ Chuẩn mật mã DES (Data encryption standard)

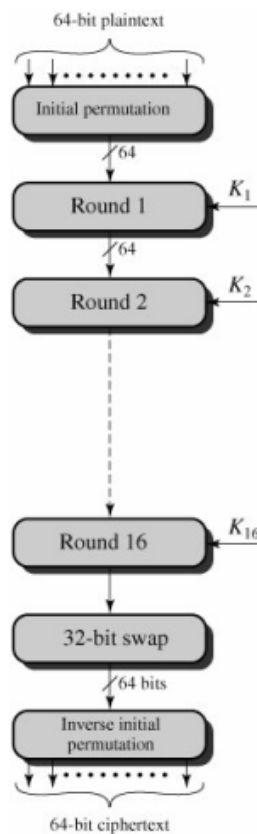
- Sơ đồ chung



- Độ dài khối : 64 bit
- Độ dài khóa: 56 bit

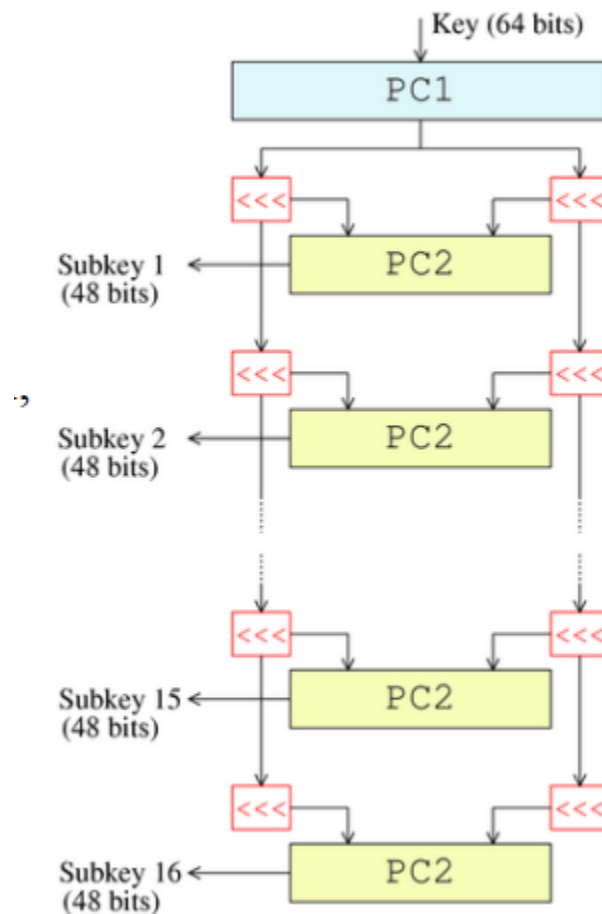
▼ Bao gồm 16 bước lặp được kẹp bởi hai toán tử hoán vị IP, IP^{-1}

- Các vòng lặp tuân theo cấu trúc Feistel
- Toán tử IP, IP^{-1} mang ý nghĩa về mặt thực tiễn hơn là thuật toán (đơn giản hóa việc “chip hóa”)
- Hàm F là nguồn gốc của sức mạnh trong DES
 - Sự lặp lại nhiều lần các bước lặp với tác dụng của F là nhằm tăng cường thêm mãnh lực của F về mặt lượng



- 16 vòng lặp của DES chạy cùng thuật toán như nhau nhưng với các khóa khác nhau, được gọi là các khóa con (Sinh ra từ khóa chính của DES bằng thuật toán sinh khóa con)
- ▼ Khóa chính K, 64 bit, qua 16 bước biến đổi, mỗi bước sinh 1 khóa con 48 bit
- Thực sự chỉ có 56 bit của khóa chính được sử dụng
 - 8 parity bits, lọc ra qua PC1.

- Các bộ biến đổi PC1 và PC2 là các bộ vừa chọn lọc vừa hoán vị
- R1 và R2 là các phép đẩy bit trái 1 và hai vị trí



Cấu trúc vòng lặp DES:

- Mỗi vòng lặp của DES thực hiện trên cơ sở công thức sau:
- $(L_i, R_i) = (R_{i-1}, L_{i-1} \oplus f(R_{i-1}, K_i))$
- Ta cũng có thể viết lại:
 - $(L_i, R_i) = T \bullet F(L_{i-1}, R_{i-1})$
 - Trong đó $F(L_{i-1}, R_{i-1}) = (L_{i-1} \oplus f(R_{i-1}, K_i), R_{i-1})$
 - T là phép đổi chỗ hai thành phần L và R
 - Tức là mỗi biến đổi vòng lặp của DES có thể coi là một tích hàm số của F và T (trừ vòng cuối cùng không có T).

- Viết lại toàn bộ **thuật toán sinh mã DES** dưới dạng công thức:

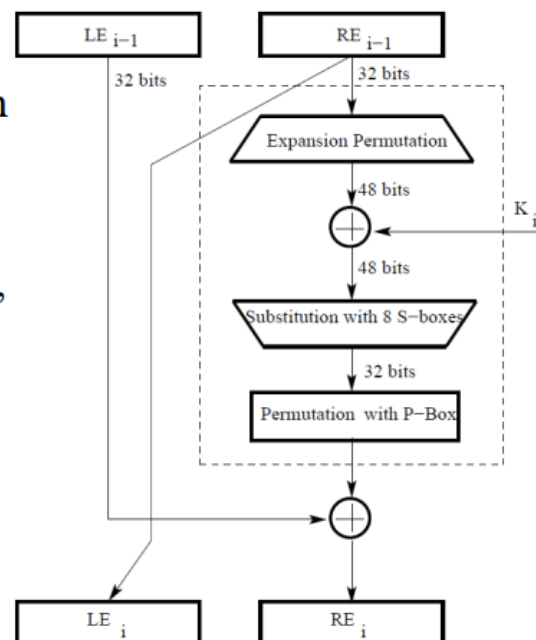
$$DES = (IP)^{-1} \bullet F_{16} \bullet T \bullet F_{15} \bullet T \bullet \dots \bullet F_2 \bullet T \bullet F_1 \bullet (IP)$$

Thuật toán giải mã DES

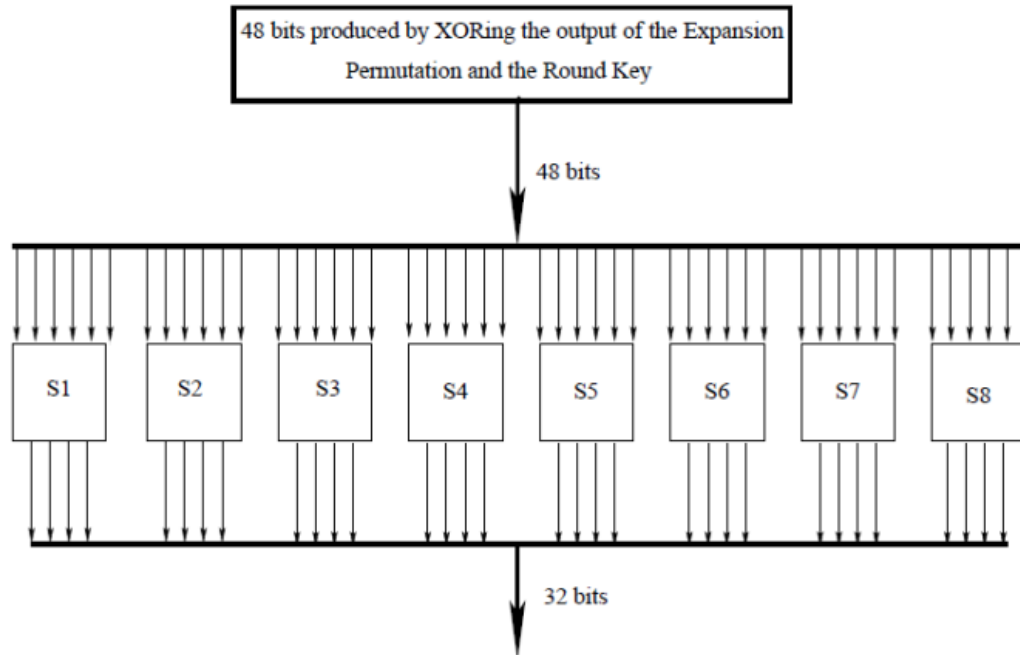
- Giống hệt như thuật toán sinh mã nhưng có các khóa con được sử dụng theo thứ tự ngược lại
- Vì vậy, thuật toán giải mã có thể được viết lại dưới dạng công thức sau:
- $DES^{-1} = (IP)^{-1} \bullet F_1 \bullet T \bullet F_2 \bullet T \bullet \dots \bullet F_{15} \bullet T \bullet F_{16} \bullet (IP)$
- Chú ý mỗi hàm T hoặc F đều là các hàm có tính chất đối hợp ($F = f^{-1}$, hay $F(F(x)) = x \rightarrow$ Thực hiện $DES^{-1} \bullet DES$ sẽ thu được phép đồng nhất.

Cấu trúc cụ thể hàm f

- 32 bit của R_{i-1} được mở rộng thành 48 bit thông qua E rồi đem XOR với 48 bit của K_i .
- 48 bit kết quả sẽ được phân thành 8 nhóm 6 bit; mỗi nhóm này sẽ qua một biến đổi đặc biệt, S-box, và biến thành 4 bit.
 - có 8 S-box khác nhau ứng với mỗi nhóm 6 bit
- 32 bit hợp thành từ 8 nhóm 4 bit (sau khi qua các S-box) sẽ được hoán vị lại theo P rồi đưa ra kết quả cuối cùng của hàm f (F_i).



S-boxes



Cấu trúc của các S-Box

- Mỗi S-box như một bộ biến đổi gồm 4 bảng biến đổi, mỗi bảng biến đổi 1 đầu vào 4 bit thành đầu ra cũng 4 bit (bảng 16 dòng).
 - Đầu vào 4 bit chính là lấy từ các bit 2-5 của nhóm 6 bit.
 - Các bit 1 và 6 sẽ dùng để xác định 1 trong 4 bảng biến đổi của S-box. Vì thế chúng được gọi là các bit điều khiển (CL và CR: left control và right control bit).

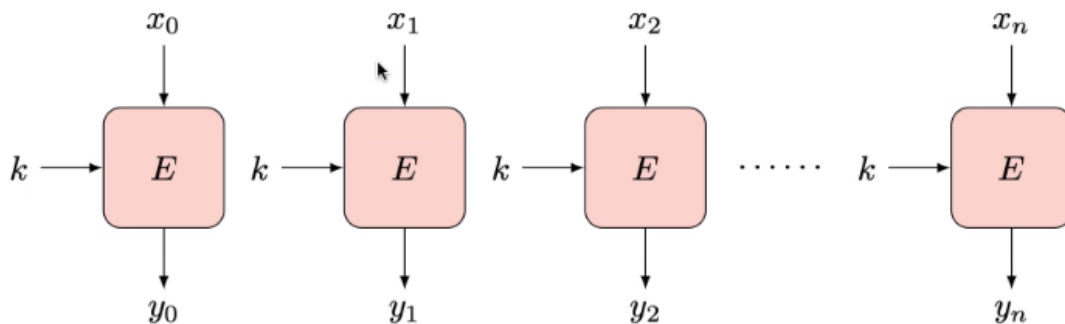
S ₅		Middle 4 bits of input															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Outer bits	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

Các chế độ mã hóa mã khối

▼ Mã khối lý tưởng

- Trên thực tế, người ta xem AES hoặc 3DES như hệ mã khối lý tưởng $E(k,x)$.
Tức là với mỗi khóa k , ánh xạ $F_k(x) = E(k, x)$ là một hoán vị ngẫu nhiên độc lập
- Làm thế nào để mã hóa thông điệp với độ dài bất kỳ ? (dùng AES hoặc 3DES)
⇒ Dùng một trong các chế độ sau:
 - "ECB" = Electronic code book
 - "CTR" = Counter mode
 - "CBC" = Cipher Block Chaining
 - "OFB" = Output Feedback
 - v.v.

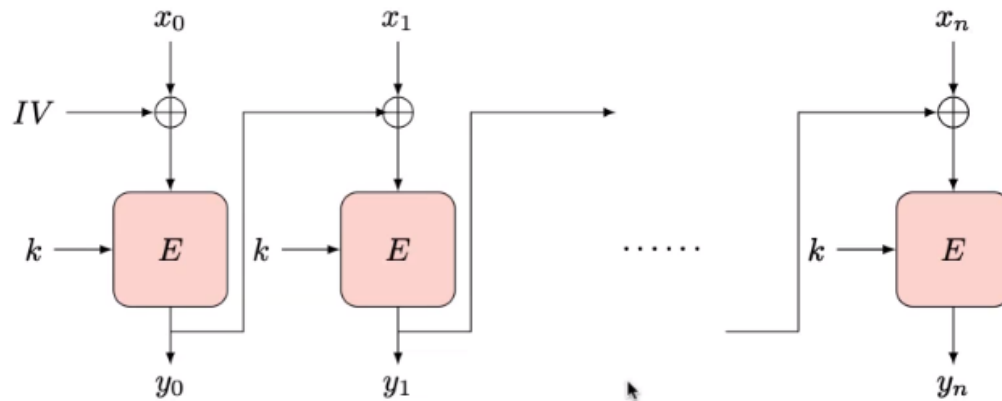
▼ Chế độ ECB (Electronic code book)



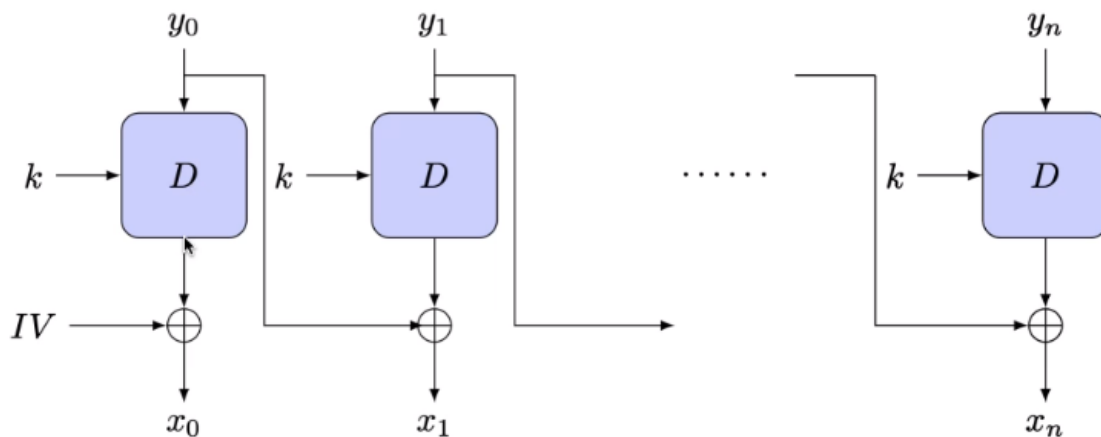
- Dữ liệu được chia thành các khối khối b bit, với b = kích thước khối.
- Với dữ liệu không chia hết cho b bit: Thêm dãy "10..0" để độ dài thông điệp chia hết cho b .
- Phép toán padding này cho có tính khả nghịch. Nó cho phép giải mã
- Nếu xuất hiện lỗi trong 1 khối mã thì chỉ 1 khối rõ bị ảnh hưởng

- ECB không an toàn. do các dữ liệu đầu vào giống nhau thì có thể được mã hóa cho đầu ra giống nhau, và người tấn công cũng có thể thay đổi, sửa hoặc tráo đổi vị trí các khối mã nếu đã biết được cấu trúc các khối

▼ Chế độ CBC (code book)



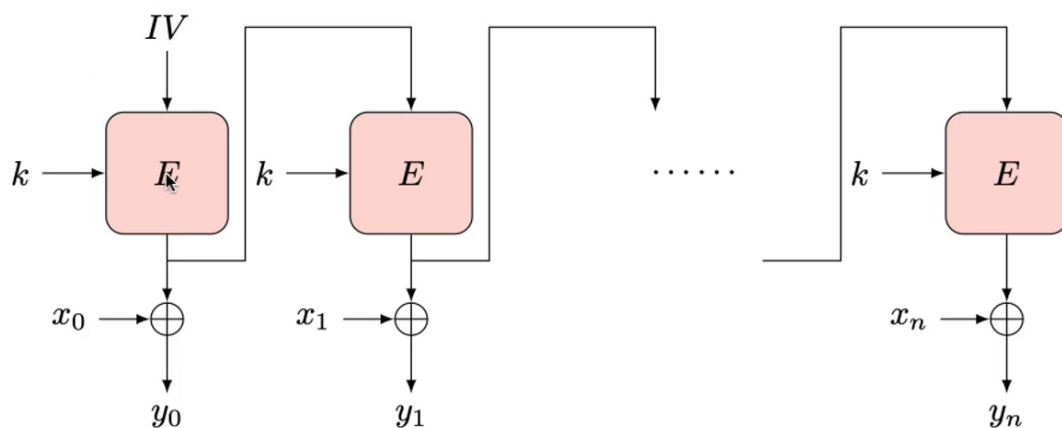
- **Thuật toán:** Chọn IV ("initialization value") một cách ngẫu nhiên, sau đó dùng y_i như " IV " cho x_{i+1} . Gửi IV cùng với bản mã. $IV || y_0 || y_1 || \dots || y_n$
- **Padding** cho CBC
 - Padding n byte, với $n > 0$
- **Ưu điểm** : Nhiều lần gửi các dữ liệu giống nhau nhưng gửi ở các lần khác nhau thì sẽ mã hóa thành nhiều bản mã khác nhau bởi vì IV thay đổi. Và đảm bảo tính làm nhiễu, làm mờ đi các đặc tính của đầu vào
- **Nhược điểm** : Không thực hiện được mã hóa song song như ECB
- **Giải mã**



- $$\begin{cases} x_0 = D(y_0) \text{ xor } (IV) \\ x_i = D(y_i) \text{ xor } (y_{i-1}) \end{cases}$$

- Khi giải mã CBC thì vẫn có thể giải mã song song, bởi vì các đầu vào độc lập
- Nếu 1 bản mã bị lỗi, sẽ ảnh hưởng tới 2 bản rõ

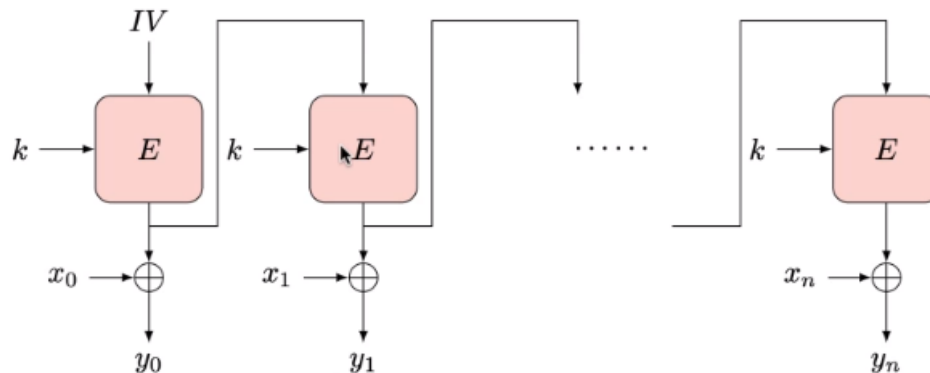
▼ Chế độ OFB(Output feedback)



- Sử dụng IV ngẫu nhiên truyền cùng bản mã
- **Ưu điểm:** Nếu người gửi biết trước IV(có thể IV là 1 số tăng dần chẳng hạn) thì người gửi có thể tính toán trước các giá trị $E(IV), E(E(IV)), \dots$ và đến lúc có các mã cần truyền thì sẽ tính toán song song ra giá trị bản mã luôn

- Không cần padding
- Công thức:
$$\begin{cases} y_0 = E(IV) \text{ xor } (x_0) \\ y_i = E^{i+1}(IV) \text{ xor } (x_i) \end{cases}$$

OFB: công thức đại số

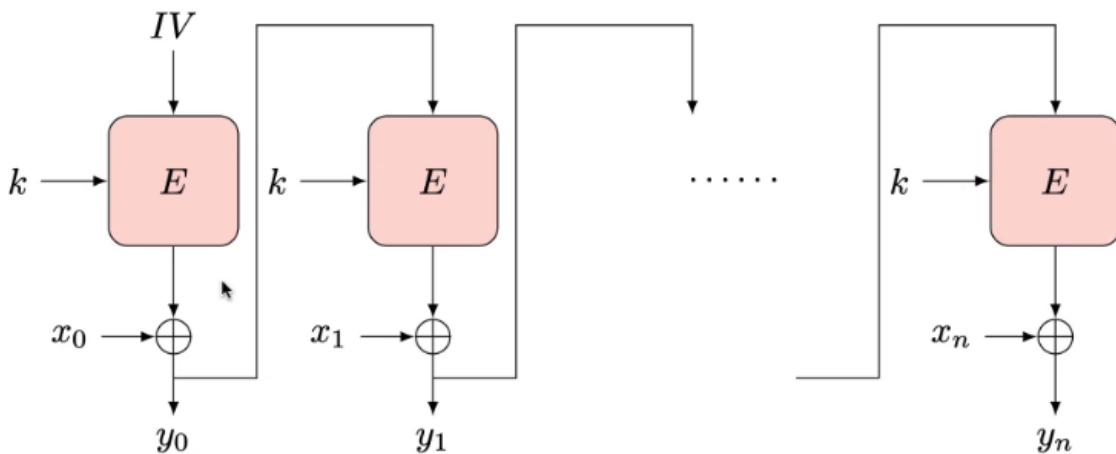


- $s_{-1} := IV$ // Khởi tạo
- $s_i := E_k(s_{i-1})$ // Khối bit giả ngẫu nhiên
- $y_i := s_i \oplus x_i$ với $i = 0, 1, 2, \dots$

- Giải mã :
$$\begin{cases} x_0 = y_0 \text{ xor } D(IV) \\ x_i = (y_i) \text{ xor } (D^{i+1}(IV)) \end{cases}$$

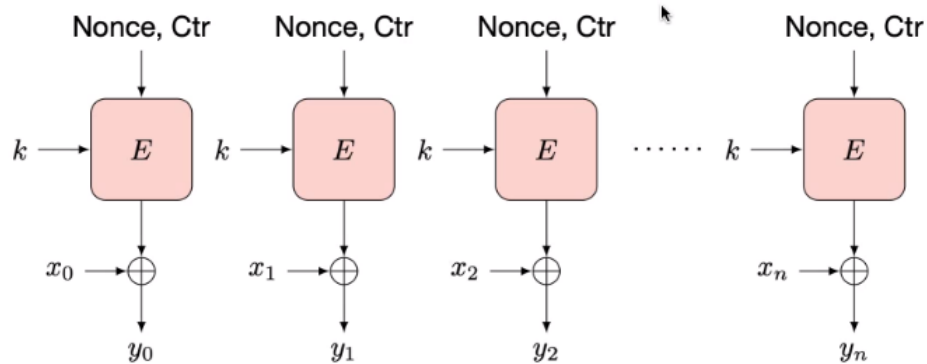
- **Ưu điểm 2** : Người nhận có thể giải mã trước các hàm E nếu biết trước IV, và khi nhận có thể giải mã song song các bản tin

▼ Chế độ CFB (Cipher feedback)



- Sử dụng IV ngẫu nhiên truyền cùng bản mã
- Công thức:
$$\begin{cases} y_0 = E(IV) \text{ xor } (x_0) \\ y_i = E(y_{i-1}) \text{ xor } (x_i) \end{cases}$$
- **Nhược điểm** : So với OFB thì thuật toán mất nhiều thời gian hơn
- **Ưu điểm**: An toàn hơn, tính hỗn loạn nhiều hơn so với OFB
- Không cần padding
- **Giải mã** :
$$\begin{cases} x_0 = y_0 \text{ xor } D(IV) \\ x_i = y_i \text{ xor } D(y_{i-1}) \end{cases}$$

▼ Chế độ CTR(Counter)



- Đảm bảo cặp **Nonce, Ctr** cặp không bao giờ lặp lại.
- Ctr được bắt đầu từ 0 cho mỗi thông điệp; và tăng (**Ctr=Ctr+1**) sau mỗi khối của thông điệp.



•

Hệ mã bất đối xứng (mã công khai)

▼ Điểm yếu của mã đối xứng

- Việc phân phối và quản lý khoá khó khăn, nhất là với các hệ thống có nhiều người sử dụng
 - N người dùng $\Rightarrow n(n-1)/2$ mối quan hệ $\Rightarrow$ mỗi người cần quản lý (n-1) khoá
- Không thể sử dụng vào chữ ký điện tử
 - Không thể đảm bảo được tính “không chối từ”

▼ Ý tưởng của Diffie-Hellman về hệ mã bất đối xứng (mã công khai)

- Về nguyên tắc, mã công khai được thiết kế trên quan điểm "hướng tới một người dùng", chứ không phải hướng tới 1 cặp người dùng (như mã bí mật)

- Mỗi người dùng tạo 2 khoá: 1 khoá giữ bí mật (secret (private) key) và 1 khoá công khai cho tất cả mọi người khác (public key)
 - Khóá bí công khai được dùng để mã hoá, khóá bí mật được dùng để giải mã

$$X = D(z, E(Z, X))$$
 - Khóá bí mật được dùng để tạo chữ ký điện tử, khóá công khai được dùng để xác thực chữ ký điện tử

$$X = E(Z, D(z, X))$$
- Mã công khai còn được gọi là mã bất đối xứng (asymmetric key cryptosystems)
 - Kể cả biết được bản mã (cipher text) và khóá công khai (public-key) thì cũng không thể tính ngược lại được bản rõ và khóá bí mật

▼ Hệ mã công khai RSA

Ý tưởng chính

- Thuật toán mã hoá và giải mã là các hàm đồng dư của các lũy thừa trong trường $Z_n = \{0, 1, 2, \dots, n-1\}$
 - Mã hoá : $Y \equiv X^e \pmod{n}$
 - Decryption: $X \equiv Y^d \pmod{n}$
 - Để đảm bảo tính đúng đắn của thuật toán
 - e & d phải thoả mãn: $X^{ed} \equiv X \pmod{n}$

- Định lý Euler: nếu $(X, n) = 1$ thì $X^{\varphi(n)} \equiv 1 \pmod{n}$
 - $\varphi(n)$: số lượng các số $k: 0 < k < n \mid (k, n) = 1$
 - Nếu $n = p \times q$ (p, q nguyên tố) $\rightarrow \varphi(n) = (p - 1)(q - 1)$
- Chọn e và tìm d sao cho $ed \equiv 1 \pmod{\varphi(n)}$
 - $d \equiv e^{-1} \pmod{\varphi(n)}$
 - $X^{ed} \equiv X^{k\varphi(n)+1} \equiv (X^{\varphi(n)})^k \times X \equiv X \pmod{n}$
- Chú ý: để giải mã được $\rightarrow$ cần biết $\varphi(n) \rightarrow$ cần biết $p, q \rightarrow$ vì n rất lớn nên việc phân tích n để tìm p, q là không khả thi

■ Tạo khoá:

- Chọn 2 số nguyên tố rất lớn và có độ lớn tương đương (~ 512 bit): p, q
- Tính $n = pq$, và $\varphi(n) = (q - 1)(p - 1)$
- Chọn 1 số tự nhiên e tùy ý, sao cho $1 < e < \varphi(n)$, và $\gcd(e, \varphi(n)) = 1$
- Tìm d , sao cho $1 < d < \varphi(n)$ và $ed \equiv 1 \pmod{\varphi(n)}$
- **Khoá công khai : (e, n) ; khoá bí mật : d**
 - Chú ý: p và q phải giữ bí mật

■ Mã hoá

- Cho trước bản rõ M biểu diễn dưới dạng nhị phân $\rightarrow$ convert M sang hệ cơ số 10: $0 < M < n$
- Dùng khoá công khai (e, n) và mã hoá:
$$C = M^e \pmod{n}$$

■ Giải mã

- Cho bản mã C , sử dụng khoá bí mật (d) và giải mã:
 - $M = C^d \pmod{n}$

■ Tính đúng đắn

- $C^d \pmod{n} \equiv M^{ed} \pmod{n} \equiv M \pmod{n}$

• Cách tính nghịch đảo đồng dư:

- Định lý Bézout: nếu $d = \text{GCD}(a, b)$ thì tồn tại 2 số x, y sao cho $d = xa + yb$ (đồng nhất thức Bézout), x, y được gọi là hệ số của a, b
- Nếu $1 = \text{GCD}(e, n) \rightarrow 1 = xe + yn \rightarrow xe \equiv 1 \pmod{n} \rightarrow x \equiv e^{-1} \pmod{n}$

Thuật toán Ôclit tìm ước số chung lớn nhất của 2 số r_0, r_1

$$\begin{aligned} r_0 &= q_1 r_1 + r_2, & 0 < r_2 < r_1 \\ r_1 &= q_2 r_2 + r_3, & 0 < r_3 < r_2 \\ &\vdots \\ r_{m-2} &= q_{m-1} r_{m-1} + r_m, & 0 < r_m < r_{m-1} \\ r_{m-1} &= q_m r_m. \end{aligned}$$

- Chứng minh được: $\text{gcd}(r_0, r_1) = \text{gcd}(r_1, r_2) = \dots = \text{gcd}(r_{m-1}, r_m) = r_m$

Cách tính đồng dư lũy thừa : PP bình phương và nhân:

■ Biểu diễn a dưới dạng nhị phân: $a = \sum_{i=0}^l a_i 2^i$

```
z ← 1
For i = l down to 0
  z ← z2 mod n
  if ai = 1 then
    z ← (z × x) (mod n)
  end if
End for
Return z
```

Tìm số dư của x^{19} khi chia cho n
 $19 = 16 + 2 + 1 = 2^4 + 2^1 + 2^0 = 10011$
 $z \leftarrow 1$
 $i = 4; a_4 = 1; z \leftarrow z^2 \times x \equiv 1^2 \times x \equiv x$
 $i = 3; a_3 = 0; z \leftarrow z^2 \equiv x^2$
 $i = 2; a_2 = 0; z \leftarrow z^2 \equiv x^4$
 $i = 1; a_1 = 1; z \leftarrow z^2 \times x \equiv x^8 \times x \equiv x^9$
 $i = 0; a_0 = 1; z \leftarrow z^2 \equiv x^{18} \times x \equiv x^{19}$

Tìm số dư của 3^{19} khi chia cho 5
 $19 = 10011$
 $z \leftarrow 1$
 $i = 4; a_4 = 1; z \leftarrow 1^2 \times 3 \equiv 3$
 $i = 3; a_3 = 0; z \leftarrow 3^2 \equiv -1$
 $i = 2; a_2 = 0; z \leftarrow (-1)^2 \equiv 1$
 $i = 1; a_1 = 1; z \leftarrow 1^2 \times 3 \equiv 3$
 $i = 0; a_0 = 1; z \leftarrow 3^2 \times 3 \equiv -3 \equiv 2$

Bài 1:

1. $28^{-1} \bmod 75$

$e = 28, n = 75$

$\text{GCD}(28, 75) = 1$

$28x + 75y = 1 \Rightarrow (x, y) = (-8, 3)$

$\Rightarrow 28^{-1} \bmod 75 = 67$

2. $17^{-1} \bmod 101$

$e = 17, n = 101$

$\text{GCD}(e, n) = 1$

$17x + 101y = 1 \Rightarrow (x, y) = (6, -1)$

$\Rightarrow 17^{-1} \bmod 101 = 6$

3. $357^{-1} \bmod 1234$

$e = 357, n = 1234$

$\text{GCD}(e, n) = 1$

$357x + 1234y = 1 \Rightarrow (x, y) = (-159, 46)$

$\Rightarrow 357^{-1} \bmod 1234 = 1075$

4. $3125^{-1} \bmod 9987$

$e = 3125, n = 9987$

$\text{GCD}(e, n) = 1$

$3125x + 9987y = 1 \Rightarrow (x, y) = (1844, -577)$

$\Rightarrow 3125^{-1} \bmod 9987 = 1844$

Nguyên tắc cấu tạo một hệ PK (trapdoor)

- Một hệ mã PKC có thể được tạo dựng trên cơ sở sử dụng một hàm kiểu one-way (1 chiều). Một hàm f được gọi là one-way nếu:
 - Đối với mọi X tính ra $Y = f(X)$ là dễ dàng.
 - Khi biết Y rất khó để tính ra X .
- Cần một hàm one-way đặc biệt, trang bị một trap-door (cửa bẫy), sao cho nếu biết trap-door này thì việc tính X khi biết $f(X)$ (tức là đi tìm nghịch đảo của f) là dễ, còn ngược lại thì khó
- Một hàm one-way có trap door như thế là một hệ mã PKC

- Lấy Ez (hàm sinh mã) là hàm one- way có trap-door
- Trap- door chính là khoá mật, mà nếu biết nó thì có thể dễ dàng tính được cái nghịch đảo của EZ tức là biết Dz, còn nếu không biết thì rất khó tính được.

▼ Hệ PKC Merkle - Hellman

Hệ PKC Merkle - Hellman

- Merkle sử dụng một mẹo là áp dụng một vector mang đặc biệt là vector siêu tăng (super-increasing)

□ thành phần $i+1$ là lớn hơn tổng giá trị của các thành phần đứng trước nó ($1 \div i$).

- Việc giải mã có thể diễn ra dễ dàng như ví dụ bằng số sau:

Vector mang siêu tăng: $a=(1,2,4,8)$

Cho $T=11$, ta sẽ thấy việc tìm $X=(X_1,X_2,X_3,X_4)$ sao cho $T= \sum a_i X_i$ là dễ dàng:

Đặt $T=T_0$

$$X_4=1 \quad T_0=T_0-X_4=3 \quad \rightarrow (X_1 \ X_2 \ X_3 \ 1)$$

$$X_3=0 \quad T_2=T_1=3 \quad \rightarrow (X_1 \ X_2 \ 0 \ 1)$$

$$X_2=1 \quad T_3=T_2-2=1 \quad \rightarrow (X_1 \ 1 \ 0 \ 1)$$

$$X_1=1 \quad \rightarrow (1 \ 1 \ 0 \ 1)$$

- Bài toán được giải quyết dần qua các bước.
 - Ở bước i , tổng đích là T_i (tức là phải tìm các a_j để tổng bằng T_i). Ta đem so sánh T_i với thành phần lớn nhất trong phần còn lại của vector, nếu lớn hơn thì thành phần này được chọn tức là X_i tương ứng bằng 1, còn ngược lại thì X_i tương ứng bằng 0. Sau đó tiếp tục chuyển sang bước sau với $T_{i+1} = T_i - X_i$.
- Cần chủ động “ngụy trang” vector siêu tăng để chỉ có người chủ mới biết còn người ngoài không thể giải mã được.

Văn K. Nguyen – Đại học

- Cơ chế ngụy trang:

- Tạo khoá:

Alice chọn một vector siêu tăng:

$$a' = (a_1', a_2', \dots, a_n')$$

a' được giữ bí mật tức là một thành phần của khoá bí mật

- Sau đó chọn một số nguyên $m > \sum a_i'$, gọi là mô-đul đồng dư và một số nguyên ngẫu nhiên ω , gọi là nhân tử, sao cho nguyên tố cùng nhau với m .
- Khoá công khai của Alice sẽ là vector a là tích của a' với nhân tử ω :

$$a = (a_1, a_2, \dots, a_n)$$

$$a_i = \omega \times a_i' \pmod{m}; i=1, 2, 3 \dots n$$

- Còn khoá bí mật sẽ là bộ ba (a', m, ω)

■ Sinh mã:

- Khi Bob muốn gửi một thông báo X cho Alice, anh ta tính mã theo công thức:

$$T = \sum a_i X_i$$

■ Giải mã:

- Alice nhận được T , giải mã như sau:

Để bỏ lớp nguy trang cô ta trước hết tính ω^{-1} (là giá trị nghịch đảo của ω , tức là $\omega \times \omega^{-1} = 1 \pmod m$, sẽ giới thiệu thuật toán tính sau), rồi tính $T' = T \times \omega^{-1} \pmod m$

- Alice biết rằng $T' = a' \cdot X$ nên cô ta có thể dễ dàng giải ra được X theo siêu tăng a' .

■ Chú thích: ở đây ta có

$$\begin{aligned} T' &= T \times \omega^{-1} = \sum a_i X_i \omega^{-1} = \sum a_i' \omega X_i \omega^{-1} \\ &= \sum (a_i' \omega \omega^{-1}) X_i = \sum a_i' X_i = a' \cdot X \end{aligned}$$

Chữ kí điện tử

▼ Yêu cầu của chữ kí điện tử

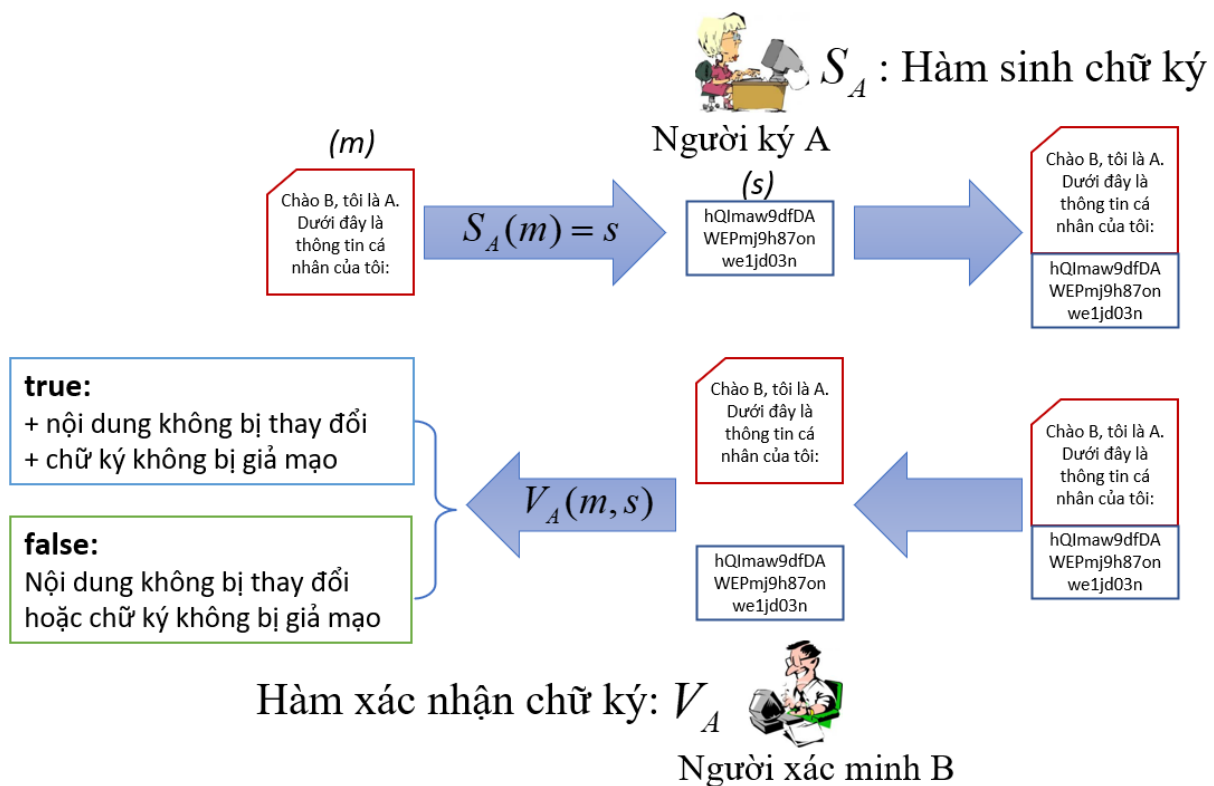
1. Xác minh được người tạo ra chữ kí
 - a. Không thể làm giả
 - b. Không thể từ chối (không phủ nhận được là chính mình đã kí)
2. Xác minh được nội dung được kí
 - a. Không thể thay đổi nội dung của bản tin đã được kí
 - b. Không thể dùng lại chữ kí cho bản tin khác

Chữ ký viết tay	Chữ ký số
Chữ ký cố định	Chữ ký thay đổi theo nội dung được ký
Gắn liền với nội dung được ký	Có thể tách khỏi nội dung được ký

▼ Tính chất của chữ kí số:

- Là một chuỗi kí tự, có nội dung phụ thuộc vào nội dung bản tin được kí
- Sử dụng thông tin mà chỉ có người kí mới có
- Gần như không thể giả mạo chữ kí

Mô hình đơn giản:



Vấn đề:

- Tốc độ chậm
- Kích thước chữ kí lớn
- Vấn đề với bản tin quá dài : Phải chia nhỏ và kí trên những bản tin nhỏ, dẫn đến có thể bị tấn công bằng cách thay đổi thứ tự gói tin, thêm bớt các gói tin

▼ **Dùng hàm băm**

Định nghĩa: Là hàm biến đổi một chuỗi kí tự có độ dài bất kì thành một chuỗi kí tự có độ dài cố định

Tính chất:

- Tính kiểm tra lỗi:
 - Thay đổi 1 bit bất kì thì sẽ cho ra đầu ra khác hoàn toàn:

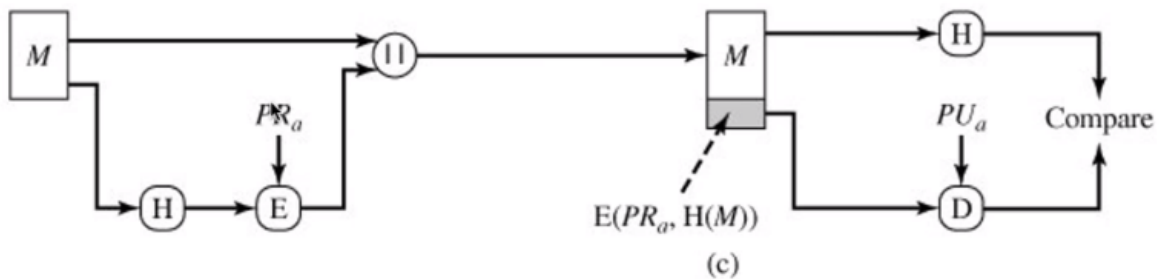
```
Message:      "A hungry brown fox jumped over a lazy dog"
SHA1 hash code: a8e7038cf5042232ce4a2f582640f2aa5caf12d2
```

```
Message:      "A hungry  brown fox jumped over a lazy dog"
SHA1 hash code: d617ba80a8bc883c1c3870af12a516c4a30f8fda
```

- Tính một chiều:
 - Biết giá trị hàm băm $\Rightarrow$ gần như ko thể suy ngược giá trị bản tin
- Tính không trùng lặp:
 - Với bản tin X cho trước > gần như không thể tìm được Y sao cho $H(x) = H(y)$ (**Tính không trùng lặp yếu**)
 - Gần như không thể tìm được (X,Y) sao cho $H(x) = H(y)$ (**Tính không trùng lặp mạnh**)

Ý tưởng chính : Kí trên giá trị hàm băm

- **Sinh chữ kí**: $s = E_{pr_A}(H(m))$



- **Xác minh chữ kí:** $D_{pb_A} eq H(m) = \begin{cases} true, if (D_{pb_A}(s) = H(m)) \\ false, if (D_{pb_A}(s) \neq H(m)) \end{cases}$

Tính đựng độ của Hàm băm:

- Việc tránh đựng độ hoàn toàn là không thể (theo lí thuyết)
 - Theo Nguyên lý Dirichlet: bỏ n+1 con thỏ vào n rọ $\Rightarrow$ ít nhất 2 con thỏ chung 1 rọ
 - Nếu thử vét cạn $|Y|+1$ bản tin $\Rightarrow$ tìm được 2 bản tin có cùng giá trị băm ($H:X \rightarrow Y$)
- **Tấn công theo kiểu nghịch lý ngày sinh:**
 - 1 năm có **365** ngày vậy nếu trong phòng có **366** người thì xác suất **100%** có 2 người trùng ngày sinh
 - Nhưng nếu chỉ cần trong phòng có **23** người, thì có đến xác suất **50%** có 2 người cùng ngày sinh, và nếu có **60** người thì có xác suất này là **90%**

$\Rightarrow$ Nếu có N giá trị có thể có của 1 chuỗi, nếu ta có số lượng $k \approx \sqrt{N}$ chuỗi sinh ra thì có 50% khả năng có 2 chuỗi giống nhau

- Cho một hàm băm $H(X)$ để mã hóa bản tin X là bản tin cần gửi $\rightarrow$ Để tấn công, kẻ tấn công phải tạo một loạt các bản tin tương tự như X ($X_1, X_2, \dots, X_k$) chỉ thay đổi 1 số chỗ để lừa người kí, sau đó sẽ tạo một nguyên mẫu bản tin giả mạo là Y, tạo các bản tin tương tự Y ($Y_1, Y_2, \dots, Y_k$) .
- $\Rightarrow$ Nhiệm vụ là tìm X_i và Y_i sao cho $H(X_i) = H(Y_i)$. Sau đó lừa người kí và lấy phần chữ kí ghép vào bản tin giả mạo để gửi cho người nhận

- Xác suất thành công trung bình của việc tìm X_i và Y_i là: $p = 1 - (1 - \frac{1}{N})^{k^2} \geq 1 - (e^{-\frac{1}{N}})^{k^2}$; $p \geq 0.5 \Leftarrow k \approx \sqrt{N}$

Quản lí và trao đổi khóa

▼ Cơ bản

Khoá phiên – khoá trao chuyển Session key - Interchange key

- Alice muốn gửi bản tin m cho Bob
 - Giả sử 2 bên sử dụng hệ mã công khai
 - Alice biết khoá công khai của Bob, Z_B
- Ví dụ về việc dùng khoá phiên
 - Alice tạo 1 khoá ngẫu nhiên k_s dùng để mã hoá bản tin m
 - Khoá này chỉ dùng để mã hoá bản tin
 - Được gọi là khoá phiên - session key
 - Alice mã hoá k_s với khoá công khai của Bob, Z_B
 - Z_B được dùng để mã hoá toàn bộ khoá phiên trao đổi giữa Alice và Bob
 - Được gọi là khoá trao chuyển - interchange key
 - Alice gửi Bob: $\{ m \} k_s \{ k_s \} Z_B$

Khóa phiên - Session key: Là khóa gắn với một phiên giao dịch. Nó có tác dụng để mã hóa thông tin chứ không dùng để xác thực chủ thể

- Tại sao lại dùng Khóa phiên ?
 - Để hạn chế lượng thông tin được mã hóa với cùng một khóa
 - Đảm bảo tính mới của khóa

- **Khóa trao chuyển - Interchange key:** Là khóa gắn với một chủ thể và dùng để xác thực chủ thể

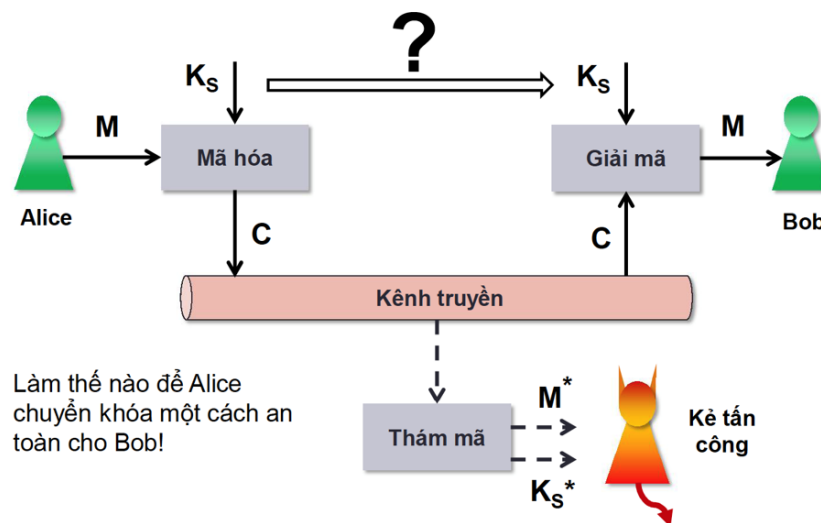
Các thuật toán trao đổi khóa

Mục tiêu: Alice, Bob thống nhất được khóa phiên (wo/ interchange key)

- Khóa phiên không thể gửi dưới dạng plain text
- Alice, Bob có thể sử dụng bên thứ 3 tin cậy
- Thuật toán cần đảm bảo an toàn với giả sử rằng toàn bộ thuật toán là public

Các giao thức trao đổi khóa sử dụng hệ mã đối xứng

Sơ đồ bảo mật sử dụng khoá đối xứng



8

▼ Giao thức trao đổi khoá không tập trung

- Khóa chính: K_M đã được A và B chia sẻ an toàn
 - Làm thế nào vì đây chính là bài toán đang cần giải quyết?
 - Khóa chính được sử dụng để trao đổi khóa phiên K_S
- Khóa phiên K_S : sử dụng để mã hóa dữ liệu trao đổi
- Giao thức 1.1
 1. $A \rightarrow B: ID_A$
 2. $B \rightarrow A: E_{K_M}(ID_B, K_S)$
 3. $(data)_{K_S}$

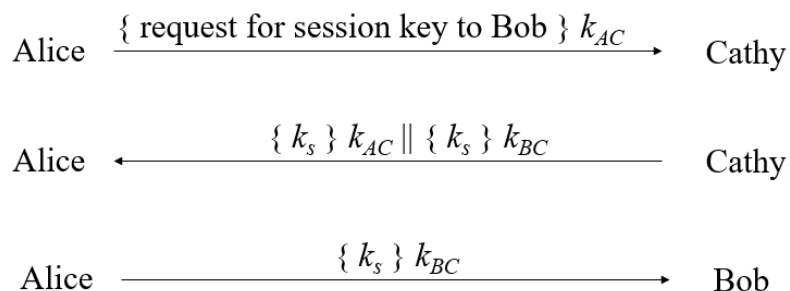
- Giao thức này đã đủ an toàn chưa?
 - Tấn công nghe lén
 - Tấn công thay thế
 - Tấn công giả mạo
 - Tấn công phát lại

⇒ Nhược điểm : Nếu thằng tấn công giải được K_S nào đó trong quá khứ, ở bước 2 nó giả mạo B và gửi khóa cho A thì nó sẽ dễ dàng dùng K_S để lừa A

- Giao thức 1.2
 - Sử dụng các yếu tố chống tấn công phát lại (replay attack)
 - $A \rightarrow B: ID_A, R_1$
 - $B \rightarrow A: E_{K_M}(ID_B, K_S, R_1, R_2)$
 - $A \rightarrow B: E_{K_S}(R_2)$
 - B: kiểm tra lại R_2
- Hạn chế của phân phối khoá không tập trung là gì?

▼ **Giao thức trao đổi khoá tập trung**

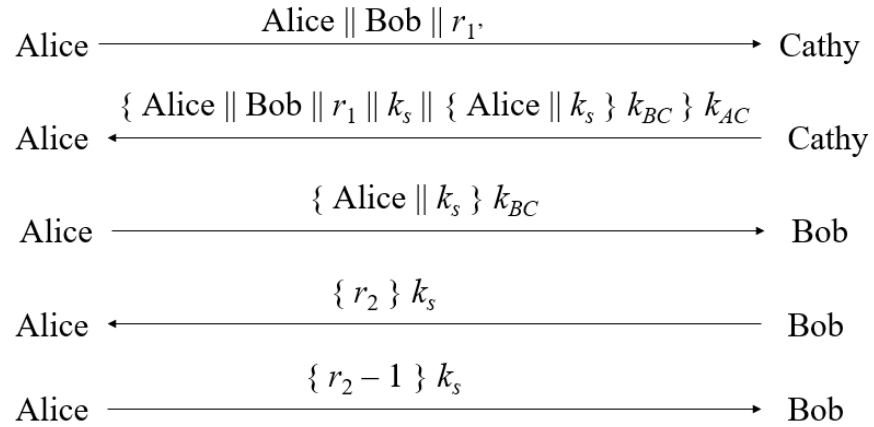
- Các thành phần tham gia
 - Alice, Bob
 - Cathy (Trọng tài): trung tâm phân phối khoá Key Distribution Center (KDC)
- Alice, Bob có khoá chung với KDC trước khi tiến hành giao thức



⇒ Nhược điểm:

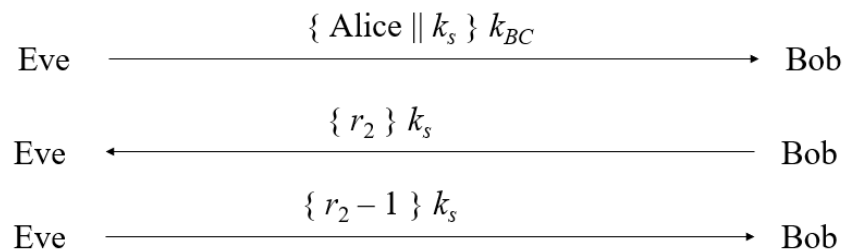
- Vấn đề
 - Làm thế nào để Bob biết người đang nói chuyện với mình là Alice?
 - Replay attack: Eve có thể bắt các gói tin, sau đó gửi lại cho Bob, Bob có thể nghĩ đó là Alice, nhưng không phải
 - Sử dụng lại khoá phiên
 - Bằng cách nào đó Eve biết được khoá phiên trước đó
 - Eve sử dụng lại các gói tin ở step 3 → Bob sử dụng lại khoá phiên
- Giao thức phải có cơ chế xác thực và chống lại replay attack

Giao thức Needham-Schroeder



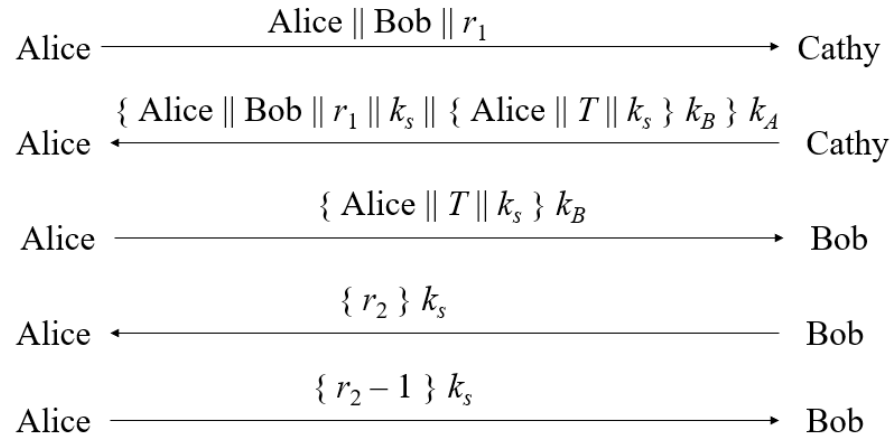
Vấn đề Denning-Sacco

- Nếu bằng cách nào đó, sau đây Eve biết được khoá k_s . Eve tấn công như sau
 - Eve giả làm Alice, giả mạo bước thứ 3



Needham-Schroeder with Denning-Sacco Modification

- Thêm timestamp



Các giao thức trao đổi khóa sử dụng hệ mã công khai

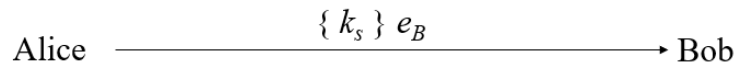
Giả thiết:

- e_A, e_B : Khóa công khai của Alice và Bob $\rightarrow$ Công khai cho tất cả mọi người
- d_A, d_B : Khóa bí mật của Alice và Bob $\rightarrow$ Chỉ Alice và Bob biết

▼ Giao thức trao đổi khoá không tập trung

- Giao thức đơn giản

- k_s : khoá phiên



- Vấn đề

- Bởi vì e_B là công khai $\rightarrow$ Bob không biết được đối phương có phải là Alice không

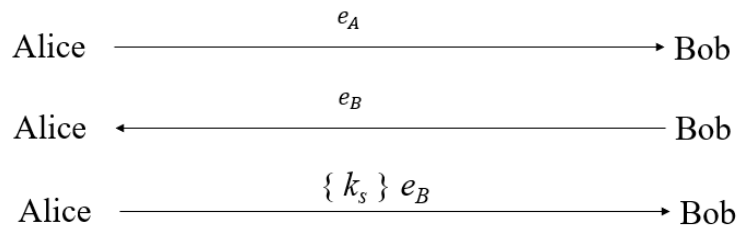
- Giải pháp đơn giản

- Sử dụng mã bí mật của Alice



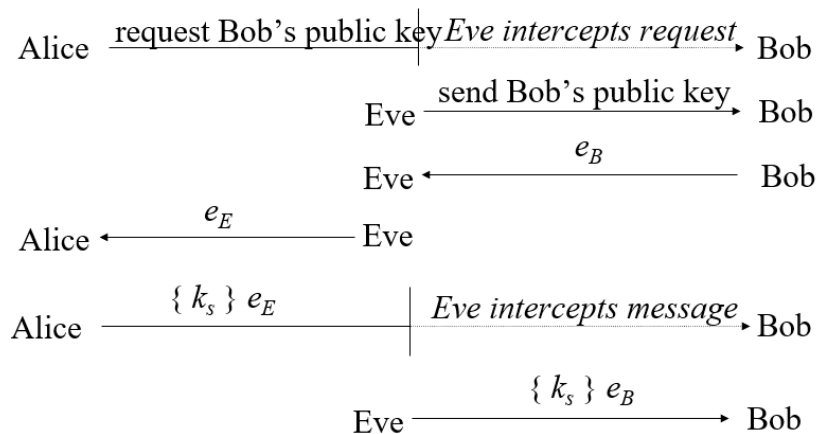
- Giao thức này có lỗ hổng nào không ?

- Nếu Alice và Bob không biết khoá công khai của nhau
 - Trước khi giao dịch, Alice và Bob gửi khoá công khai trực tiếp cho nhau



Tấn công kẻ ngồi giữa Man-in-the-middle attack

- Eve giả vờ là Bob khi nói chuyện với Alice, giả vờ là Alice khi nói chuyện với Bob

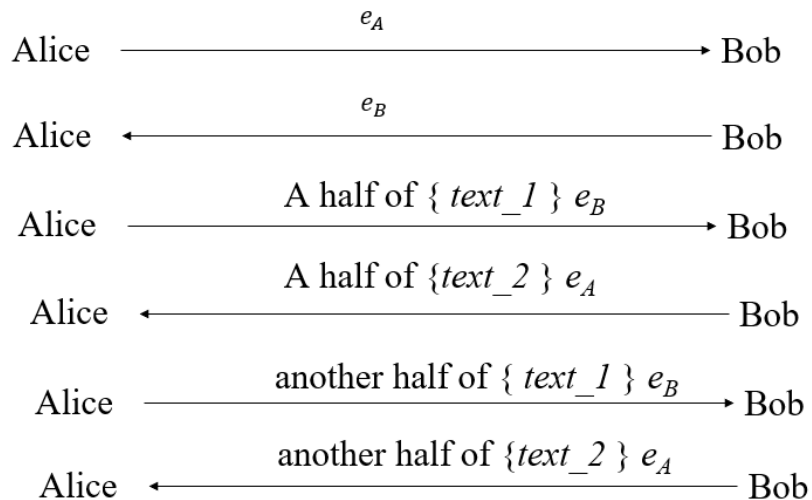


24

Chống tấn công kẻ ngồi giữa

- Interlock: đề xuất bởi Ron Rivest and AdiShamir
 1. Alice gửi khoá công khai của Alice cho Bob
 2. Bob gửi khoá công khai của Bob cho Alice
 3. Alice mã hoá gói tin của mình bằng khoá công khai của Bob. Alice gửi 1 nửa bản mã cho Bob
 4. Bob mã hoá gói tin của mình bằng khoá công khai của Alice. Bob gửi 1 nửa bản mã cho Alice
 5. Alice gửi nửa còn lại của bản mã cho Bob
 6. Bob ghép 2 nửa bản mã lại, giải mã dùng khoá bí mật của Bob. Nếu bản mã được giải mã thành công Bob gửi lại nửa bản mã còn lại ở step 4 cho Alice
 7. Alice ghép 2 nửa bản mã nhận được từ Bob và giải mã
 8. Câu hỏi
 1. Tại sao interlock chống được man in the middle attack

Solution for Man-in-the-Middle Attack



- ▼ Giao thức trao đổi khoá tập trung

- Sử dụng bên thứ 3 tin cậy – PKA (Public Key Authority)
 - Có cặp khóa (e_{PKA}, d_{PKA})
 - Có công khai của A (e_A) và B (e_B)
 - A và B đều có khóa công khai e_{PKA} của PKA
- Giao thức 1
 1. $A \rightarrow PKA: Alice \parallel Bob$
 2. $PKA \rightarrow A: \{Bob \parallel e_B\}d_{PKA}$
 3. $A \rightarrow B: \{r_1\}e_B$
 4. $B \rightarrow PKA: Alice \parallel Bob$
 5. $PKA \rightarrow B: \{Alice \parallel e_A\}d_{PKA}$
 6. $B \rightarrow A: \{r_1\}e_A$
- Câu hỏi
 - Kiểm tra tính an toàn của giao thức 1
 - Hạn chế của giao thức này?
- Giao thức 2: Bên thứ 3 được tin cậy – CA(Certificate Authority)
 - Có cặp khóa (e_{PKA}, d_{PKA})
 - Phát hành chứng thư số cho khóa công khai của các bên có dạng
 - $Cert_A = \{ID_A \parallel e_A \parallel Time_A\}d_{PKA}$
 - ID_A : định danh của thực thể A
 - e_A : khóa công khai của thực thể A đã được đăng ký tại CA
 - $Time_A$: Thời hạn sử dụng khóa công khai. Thông thường có thời điểm bắt đầu có hiệu lực và thời điểm hết hiệu lực
- Giao thức 2: Bên thứ 3 được tin cậy – CA(Certificate Authority)
 1. $A \rightarrow CA: ID_A \parallel e_A \parallel Time_A$
 2. $CA \rightarrow A: Cert_A = \{ID_A \parallel e_A \parallel Time_A\}d_{PKA}$
 3. $B \rightarrow CA: ID_B \parallel e_B \parallel Time_B$
 4. $CA \rightarrow B: Cert_B = \{ID_B \parallel e_B \parallel Time_B\}d_{PKA}$
 5. $A \rightarrow B: Cert_A$
 6. $B \rightarrow A: Cert_B$
- Câu hỏi
 - Cải tiến giao thức trên nhằm tăng cường tính an toàn
 - Gợi ý: dùng các phương pháp kiểm tra tính toàn vẹn, ...

- Giao thức 2
 1. $A \rightarrow PKA: Alice \parallel Bob \parallel T_1$ (T_1 : timestamp)
 2. $PKA \rightarrow A: \{Bob \parallel e_B\}d_{KPA}$
 3. $A \rightarrow B: \{r_1\}e_B$
 4. $B \rightarrow PKA: Alice \parallel Bob \parallel T_2$ (T_2 : timestamp)
 5. $PKA \rightarrow B: \{Alice \parallel e_A\}d_{KPA}$
 6. $B \rightarrow A: \{r_1\}e_A$
- Ý nghĩa của T_1 và T_2
 - chống tấn công phát lại
- Câu hỏi: giao thức này có hạn chế gì?

Các mô hình quản lý truy cập

Access Control (AC) - Chức năng của nó là quản lý các vai trò (người, máy, tiến trình,...) nào có quyền truy cập vào tài nguyên nào của hệ thống (file nào có thể đọc, chương trình nào có thể chạy,...)

- AC cực kì phổ biến:
 - **Trong lớp ứng dụng:** AC xuất hiện ở hầu khắp các ứng dụng liên quan đến doanh nghiệp
 - **Lớp trung gian:** AC có trong các hệ cơ sở dữ liệu
 - **Hệ điều hành:** AC điều khiển truy cập đến tệp tin và cổng
 - **Phần cứng:** AC bảo vệ bộ nhớ, các mức đặc quyền
- ▼ **Mô hình ma trận điều khiển truy cập (ACM)**

Access control Matrix

- **S**: tập hợp các chủ thể S
- **O**: Tập hợp các đối tượng
- **R**: Tập hợp các quyền
- Ma trận điều khiển truy cập
 - Mỗi chủ thể ứng với một dòng
 - Mỗi đối tượng/chủ thể ứng với một cột
 - Mỗi ô liệt kê quyền mà chủ thể có thể sử dụng đối với đối tượng/chủ thể

		Đối tượng/chủ thể					
		o_1	...	o_m	s_1	...	s_n
Chủ thể	s_1						
	s_2						
	...						
	s_n						

- Tập chủ thể $S = \{ s_1, \dots, s_n \}$
- Tập đối tượng $O = \{ o_1, \dots, o_m \}$
- Tập quyền $R = \{ r_1, \dots, r_k \}$
- Ô $A[s_i, o_j] \subseteq R$
- $A[s_i, o_j] = \{ r_x, \dots, r_y \}$ có nghĩa chủ thể s_i có các quyền $r_x, \dots, r_y$ đối với đối tượng o_j

Cách cài đặt

- Lưu trữ ma trận điều khiển truy cập
 - **Phân rã theo dòng**: danh sách khả năng (danh sách năng lực)
 - Tất các truy nhập đến các đối tượng hay chủ thể khác được thực hiện thông qua năng lực truy nhập của **chủ thể**
 - **Phân rã theo cột**: danh sách điều khiển truy cập (ACL)
 - ACL là danh sách các quyền gắn với một **đối tượng**
 - **Biểu diễn gián tiếp**:
 - Ví dụ: danh sách key và lock

- Ví dụ: nhóm, vai trò, nhiều mức gián tiếp biểu diễn gián tiếp và nhiều khóa
- Việc thực hiện biểu diễn gián tiếp một cách chính xác và tiện lợi chính là chìa khóa để quản lý điều khiển truy cập.

▼ Mô hình điều khiển truy cập tùy nghi (DAC)

- **Discretionary access control**
- **Điều khiển truy cập tùy nghi (DAC)**
 - Quyền truy nhập (right) cho từng cặp (chủ thể, đối tượng) có thể xác định riêng rẽ và có thể quyết định bởi chủ nhân của đối tượng (khác với **Mandatory Access Control**, được quyết định bởi chính sách hệ thống)
 - **Không** có định nghĩa **chặt chẽ**
 - Sử dụng **phổ biến** trong các **hệ điều hành** hiện đại
 - **Gắn liền** với sự sử dụng khái niệm **chủ nhân** của mỗi đối tượng
 - **Chủ nhân kiểm soát khả năng truy nhập** của các chủ thể khác đối với đối tượng
 - Cho phép **cấp phát quyền truy cập** đến các chủ thể khác
- **Nhược điểm:**
 - DAC không thể ngăn chặn:
 - Trojan horse
 - Malware
 - Bug phần mềm
 - Người dùng cục bộ cố ý gây thiệt hại
 - DAC không thể kiểm soát luồng thông tin
- Hoạt động của **HRU**
 - Hình thức hóa ma trận điều khiển truy nhập, hướng đến mô tả các hoạt động của hệ điều hành

- Cung cấp một mô hình, đủ mạnh để mã hóa một vài cách thức điều khiển truy nhập, và đủ chặt chẽ để có thể phân tích các thuộc tính về an toàn bảo mật
- Giới thiệu bài toán An toàn “safety problem”
- Chứng minh rằng bài toán an toàn chỉ có thể giải được khi đưa về những trường hợp đặc biệt với một số điều kiện đủ tốt

Các thao tác cơ bản:

- **create subject s; create object o**
 - Tạo mới dòng, cột trong ACM; tạo mới một cột trong ACM
- **destroy subject s; destroy object o**
 - Xóa dòng, cột của ACM; xóa cột của ACM
- **enter r into A[s, o]**
 - Thêm vào quyền r cho chủ thể s đối với đối tượng o
- **delete r from A[s, o]**
 - Thu hồi quyền r khỏi chủ thể s đối với đối tượng o
- Tạo tệp dữ liệu:
- Tiến trình p tạo tệp dữ liệu mới f với các quyền r và w

```
command create_file(p, f)
    create object f;
    enter own into A[p, f];
    enter r into A[p, f];
    enter w into A[p, f];
end
```

▼ Mô hình điều khiển truy cập cường chế (MAC)

- **Mandatory access control**
- **Đối tượng:** Phân loại mức bảo mật
 - ví dụ: grades=(confidential, {student-info})
- **Chủ thể:** Nhãn bảo mật
 - Ví dụ: Joe = (confidential, {student-info})
- **Luật truy nhập:** Định nghĩa bằng cách so sánh mức độ mật của đối tượng yêu cầu với nhãn bảo mật của chủ thể
 - Ví dụ: Chủ thể có thể đọc đối tượng chỉ khi label(chủ thể) cao hơn label(đối tượng)
- **Cấp bậc của chủ thể (nhãn) có dạng: (A,C)**
 - A - cấp, mức
 - C - nhóm, thể loại truy cập
 - $\Rightarrow$ Nếu **nhãn chủ thể có cấp cao hơn** cấp của nhãn đối tượng **nhưng nhóm của nhãn lại không bao gồm** nhóm của nhãn đối tượng thì cũng **ko truy cập** dc đối tượng đó ^^
- **Mô hình Bell-Lapadula (BLP)**
 - Bảo vệ tính mật
 - Hỗ trợ quản trị phi tập trung
- Các luật cơ bản của BLP:
 - Bảo mật đơn giản: Một chủ thể s sẽ chỉ được phép thực hiện thao tác đọc đối với đối tượng o nếu nhãn của s là ưu thế hơn nhãn của o
 - No read up
 - Áp dụng cho **tất cả chủ thể**
 - Chủ thể s được đọc đối tượng o nếu và chỉ nếu $L(o) \leq L(s)$ và s được cấp phép đọc đối với o (Kết hợp của điều khiển truy cập cưỡng chế (quan hệ của mức độ bảo mật) và điều khiển truy cập tùy nghi (yêu cầu cấp phép))

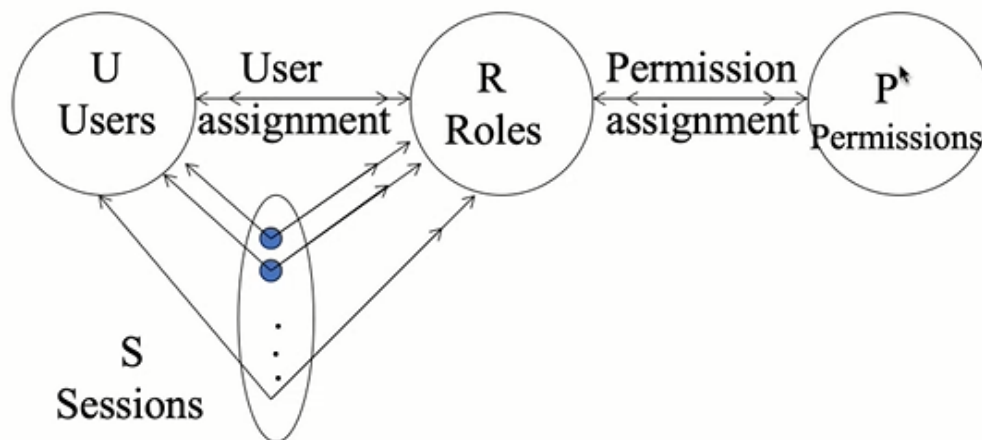
- Một chủ thể s chỉ được thực hiện thao tác viết lên đối tượng o khi nhãn của o là ưu thế hơn nhãn của s
 - No write down
 - Chỉ áp dụng với **chủ thể không được coi là tin cậy**
 - Chủ thể s có thể viết lên đối tượng o nếu và chỉ nếu $L(s) \leq L(o)$ và s được cấp phép để viết lên o
- **Ưu điểm:**
 - Rất bảo mật
 - Mang tính bắt buộc tập trung
- **Khuyết điểm**
 - Có thể quá hạn chế
 - Cần các cơ chế bổ sung để triển khai hệ thống bảo mật đa cấp
 - Quản trị bảo mật là rất khó

▼ **Mô hình RBAC**

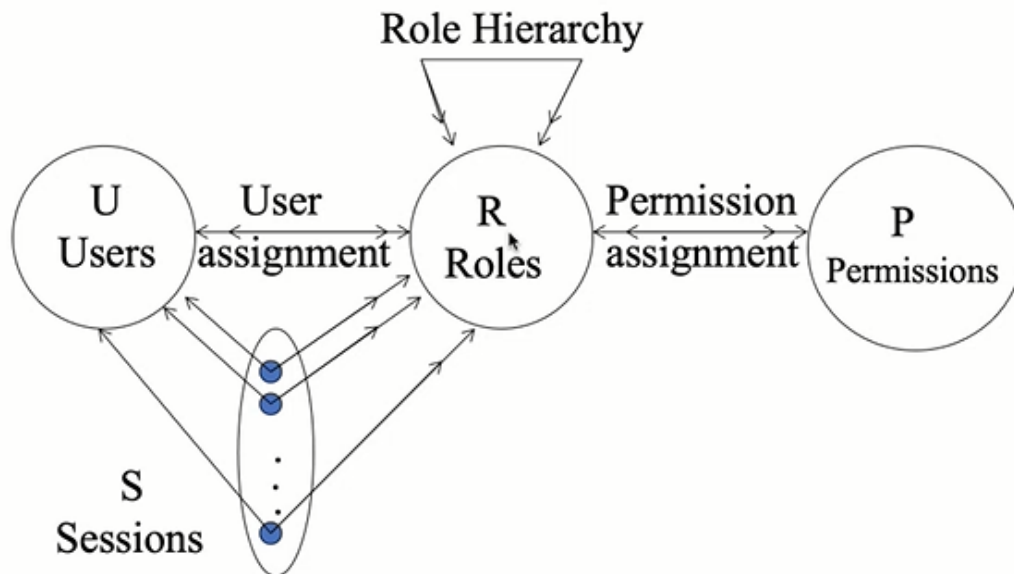
- **Role-Based access control**
- Các hệ thống đa người dùng
- Các hệ thống đa ứng dụng
- Gắn quyền với vai trò
- Sự gắn quyền với vai trò thường là lâu dài
- Gần với trực giác: Năng lực, nghĩa vụ và trách nhiệm gắn liền với nhau
- Phù hợp với mô hình doanh nghiệp:
 - Chia sẻ trách nhiệm
 - Ủy quyền
- Linh hoạt: dễ dàng thay đổi để phù hợp với các yêu cầu bảo mật mới
- Hỗ trợ:

- Đặc quyền tối thiểu
- Chia sẻ trách nhiệm
- Khái quát hóa thông tin dữ liệu
- **Vai trò:**
 - **Nhóm người dùng:** Tập người dùng, mỗi người dùng có thể có các quyền khác nhau
 - **Vai trò:** Khái niệm trung gian giữa tập các người dùng và một tập các quyền khai thác
 - **RBAC** độc lập với DAC và MAC (có thể cùng tồn tại)

RBAC₀



RBAC₁



An ninh mạng

▼ Mạng không dây

- Mạng không dây **hiều rủi ro hơn** mạng có dây vì trong vòng bán kính xung quanh các thiết bị phát sóng đều có thể nhận được các gói tin gửi đi (**Không có sự bảo mật về mặt vật lý**). Còn mạng có dây thì phải kết nối vật lý vào mạng LAN cục bộ mới có thể nhận được các gói tin trong mạng LAN đó. ⇒ **Rất dễ bị nghe lén gói tin**
- Mạng không dây thì việc phát gói tin từ máy này sang bất kỳ máy nào khác là dễ dàng vì thế **rất dễ để truyền những gói tin độc hại đến các máy tính kết nối wifi** nhằm DoS, giả mạo gói tin, quấy nhiễu,....
- ⇒ Cần một **cơ chế bảo mật đi kèm với các giao thức truyền tin**, với các yếu tố:

- Tính xác thực (Authenticity)
- Tính mật (Confidentiality)
- Tính toàn vẹn (Integrity)
- Tính điều khiển truy cập (Access control)
- Tính phát hiện gói tin lặp (Replay detection)
- Tính tránh tắc nghẽn (Protection against jamming)
- Với giao thức 802.11: WEP, WEP2, WPA, WPA2
- Điểm yếu của WEP
 - Máy trạm không biết AP có phải giả mạo hay không vì không phải xác thực 2 chiều
 - Không dùng khóa phiên mà dùng trực tiếp khóa bí mật để mã hóa dữ liệu
⇒ Dễ bị bẻ khóa

Các cách tấn công vào mạng không dây:

- **Tấn công giả mạo:**
 - AP (giả mạo) → STA : r
 - STA → AP (giả mạo): IV | $r \oplus K$ (K là khóa)
 - Kẻ giả mạo sẽ tính được $r \oplus (r \oplus K) = K$
 - Từ đó kẻ giả mạo sẽ có thể giao tiếp với STA một cách dễ dàng do đã có khóa K
- **Tấn công chỉnh sửa bản rõ (tính toàn vẹn)**

- ▶ The attacker can manipulate messages despite the ICV mechanism and encryption

- ▶ CRC is a linear function to XOR:

$$\text{CRC}(X \oplus Y) = \text{CRC}(X) \oplus \text{CRC}(Y)$$

- attacker observes $(M \parallel \text{CRC}(M)) \oplus K$ where K is the RC4 output
- for any ΔM , the attacker can compute $\text{CRC}(\Delta M)$
- hence, the attacker can compute:

$$\begin{aligned} M' &= M \oplus \Delta M \\ \rightarrow (M' \parallel \text{CRC}(M')) \oplus K &= ((M \oplus \Delta M) \parallel \text{CRC}(M \oplus \Delta M)) \oplus K \\ &= ((M \oplus \Delta M) \parallel (\text{CRC}(M) \oplus \text{CRC}(\Delta M))) \oplus K \\ &= ((M \parallel \text{CRC}(M)) \oplus K) \oplus (\Delta M \parallel \text{CRC}(\Delta M)) \end{aligned}$$

- Kẻ tấn công sửa cả plain text và CRC khiến người dùng ko nhận ra

- **Tấn công tái sử dụng IV (tính mật)**

- Sử dụng mỗi chuỗi IV cho mỗi thông điệp, mà không gian IV khá nhỏ - 24 bits $\Rightarrow 2^{24}$ khả năng.
- Theo 802.11 $\Rightarrow$ IV có thể bị lặp lại trong 9h gửi tin liên tục
- 2 bản mã có cùng IV có thể bị lộ cả 2 thông tin cùng lúc bằng cách:

$$\begin{aligned} \text{if } C_1 &= P_1 \oplus \text{RC4}(IV, k) \text{ and } C_2 = P_2 \oplus \text{RC4}(IV, k) \\ \text{then } C_1 \oplus C_2 &= (P_1 \oplus \text{RC4}(IV, k)) \oplus (P_2 \oplus \text{RC4}(IV, k)) = P_1 \oplus P_2 \end{aligned}$$

- **Tấn công khai thác RC4 (tính mật)**

- Đối với khóa K - WEP key là khóa yếu
- Cipher = plain $\oplus$ keystream
- $\Rightarrow$ keystream = plain $\oplus$ Cipher
- **Một số byte đầu tiên của bản tin chuẩn 802.11 là cố định theo chuẩn**
- Có 1 số vector IV sẽ tạo ra keystream yếu : Biết byte thứ $m \Rightarrow$ suy ra byte thứ $m+1$

- $\Rightarrow$ Chỉ cần vài byte đầu tiên của plaintext (tuân theo chuẩn) xor với vài byte đầu của cipher text $\Rightarrow$ suy ra dc keystream
- Các chuyên gia khuyến cáo bỏ 256 bytes đầu của keystream nhưng WEP ko có điều này

Tấn công các giao thức định tuyến

BGP: Giả mạo thông tin định tuyến để điều hướng dữ liệu. Hậu quả: tấn công từ chối dịch vụ, man-in-the-middle

▼ BGP hijacking

- Thay đổi bảng định tuyến BGP
- Kẻ tấn công tạo ra BGP router nào đấy và gửi gói tin broadcast đi các network xung quanh tuyên bố nó sở hữu 1 dãy ip nào đấy, và khi các network gửi gói tin đến thì nó sẽ nghe lén sau đấy mới chuyển đến địa chỉ ip thật hoặc chỉ đơn giản là ăn cắp luôn gói tin.
- Hậu quả
 - Tăng thời gian truyền
 - Định hướng tới những nơi không mong muốn (fake website)
 - Nghe lén,....

Tấn công can thiệp vào kết nối TCP

- Quá trình trao đổi dữ liệu kết thúc bình thường: giao thức TCP cho phép 2 bên đóng liên kết một cách độc lập (gửi gói tin FIN)
 - Tin cậy: chờ nhận ACK
 - Liên kết chỉ thực sự hủy khi 2 bên đã đóng
- Ngược lại, nếu quá trình trao đổi dữ liệu không thể kết thúc bình thường (tiến trình ứng dụng kết thúc đột ngột, các gói tin lỗi), gói tin RST (reset) được gửi đi:
 - Việc đóng liên kết xuất phát từ một bên
 - Không cần chờ ACK
 - Liên kết được hủy nếu Sequence Number là phù hợp

- ⇒ kẻ tấn công có thể ngắt kết nối đột ngột của người dùng nếu biết được thông tin về số hiệu cổng, Sequence Number.
- **RST Injection**
- **Data Injection**

Tấn công giả mạo kết nối TCP

Ôn tập :

 Ôn tập đại cương ATTT